

KPI Engine Prompts

In our TPM solution KPI Engine must be more dynamic, it should work as an engine; KPIs with formulas that calculate automatically. I will share necessary informations with you, please check all and make the developments and changes.

REQUIREMENTS:

- Admins define KPIs in a UI with formulas stored as text
- Formulas reference other KPIs by code (e.g., "PLANNED_VOL - BASE_VOL")
- System must calculate KPIs in correct dependency order
- Support different formula types: expression, conditional, javascript
- Track what each KPI depends on for cascade calculations
- Support aggregation methods (sum, avg, min, max) when rolling up SKU to FU level

TECH STACK:

- Supabase (PostgreSQL)
- Need Row Level Security (RLS) policies
- Use UUIDs for primary keys
- Use JSONB for array fields

CREATE THESE TABLES:

1. **kpis** - Main KPI definitions Fields needed:

id (UUID, primary key) kpi_code (VARCHAR(50), unique) - e.g., 'INCR_VOL', 'GP_ROI_PCT' kpi_name (VARCHAR(200)) - display name kpi_group (VARCHAR(100)) - e.g., 'Volume', 'Profit', 'ROI' formula_type (VARCHAR(50)) - 'expression', 'conditional', 'user_input', 'external' formula_text (TEXT) - the actual formula depends_on_kpis (JSONB) - array of KPI codes this depends on calculation_order (INTEGER) - for sorting calculation sequence calculation_level (VARCHAR(50)) - 'sku', 'fu', 'plan' display_format (VARCHAR(50)) - 'number', 'currency', 'percentage' decimal_places (INTEGER) show_in_grid (BOOLEAN) aggregation_method_fu (VARCHAR(20)) - 'sum', 'avg', 'min', 'max' is_active (BOOLEAN) created_at, updated_at timestamps

2. **promo_mechanics** - Tactic definitions (what planners can input) Fields needed:

id, mechanic_code, mechanic_name category ('on_invoice_discount', 'off_invoice_discount', 'per_unit_support', 'lumpsum') spending_type ('on_invoice', 'off_invoice') input_type ('percentage', 'currency', 'units') min_value, max_value, default_value calculation_formula (TEXT) - how to calculate spend from input applicable_channels (JSONB) - array of channels applicable_categories (JSONB) - array of categories show_in_grid (BOOLEAN) is_active (BOOLEAN)

3. plan_sku_volumes - Store user inputs and calculated values

id, plan_id, sku_id base_volume (DECIMAL) - from historical data planned_volume (DECIMAL) - user input calculated_kpis (JSONB) - store all calculated KPI values updated_at

4. plan_mechanic_values - Store tactic inputs at FU level

id, plan_fu_id, mechanic_id entered_value (DECIMAL) - user input calculated_spend (DECIMAL) - result of formula updated_at

ALSO PROVIDE:

1. Indexes for performance:

Index on kpis.kpi_code Index on kpis.calculation_order Index on promo_mechanics.mechanic_code Index on plan_sku_volumes (plan_id, sku_id)

2. Sample INSERT statements for these KPIs:

LEVEL 1 (User Input - Order 1-10):

- PLANNED_VOL (user_input)
- BASE_VOL (external)
- BPTT (external - from master data)
- COGS (external - from master data)
- CPP_ON_PCT (user_input - tactic)

LEVEL 2 (Simple calculations - Order 11-20):

- $INCR_VOL = PLANNED_VOL - BASE_VOL$
- $VOL_UPLIFT_PCT = (INCR_VOL / BASE_VOL) * 100$
- $BASE_GSV = BASE_VOL * BPTT$
- $PLANNED_GSV = PLANNED_VOL * BPTT$
- $INCR_GSV = PLANNED_GSV - BASE_GSV$

LEVEL 3 (Dependent calculations - Order 21-30):

- $CPP_ON_SPEND = (PLANNED_GSV - PLANNED_LTA_ON) * (CPP_ON_PCT / 100)$
- TOTAL_PLANNED_SPEND = sum of all spends

LEVEL 4 (Profit calculations - Order 31-40):

- $BASE_COGS = BASE_VOL * COGS$
- $PLANNED_COGS = PLANNED_VOL * COGS$
- $BASE_GP = BASE_TO - BASE_COGS$
- $PLANNED_GP = PLANNED_TO - PLANNED_COGS$
- $INCR_GP = PLANNED_GP - BASE_GP$

LEVEL 5 (ROI - Order 41-50):

- $GP_ROI_PCT = (INCR_GP / INCR_SPEND) * 100$
- RAG_STATUS (conditional) = IF(GP_ROI_PCT >= 20, 'GREEN', IF(GP_ROI_PCT >= 10, 'AMBER', 'RED'))

3. Sample promo_mechanics records:

- CPP On-Invoice % (percentage, 0-100, applies to all channels/categories)
- Visibility Lumpsum MT/PH (currency, \$0-\$50000, applies to NKA channel only)
- Price Support per Unit (currency, \$0-\$5, applies to all)

FORMAT: Provide complete SQL migration file ready to run in Supabase SQL editor.

--- ## **PROMPT 2: Formula Parser Service**

Create a TypeScript service that parses formula strings and executes them safely.

CONTEXT: I'm building a dynamic KPI system where formulas are stored as text in the database like:

- "PLANNED_VOL - BASE_VOL"
- "(INCR_GP / INCR_SPEND) * 100"
- "IF(GP_ROI_PCT >= 20, 'GREEN', IF(GP_ROI_PCT >= 10, 'AMBER', 'RED'))"

REQUIREMENTS:

1. Extract Dependencies - Automatically find variable names in formula

Input: "(PLANNED_VOL - BASE_VOL) / BASE_VOL * 100" Output: ["PLANNED_VOL", "BASE_VOL"] Rules: Variable names are UPPERCASE with underscores, exclude function names like IF, SUM, AVG

2. Parse Expression Formulas - Simple math operations

Support: +, -, *, /, (), parentheses Replace variable names with actual values from context Example: "PLANNED_VOL - BASE_VOL" with {PLANNED_VOL: 4200, BASE_VOL: 3200} returns 1000

3. Parse Conditional Formulas - IF statements

Syntax: IF(condition, true_value, false_value) Support nested IFs Support comparisons: >=, <=, >, <, ==, != Example: "IF(GP_ROI_PCT >= 20, 'GREEN', 'RED')"

4. Safe Evaluation - No arbitrary code execution

Use Function constructor with whitelist Only allow: numbers, basic operators, parentheses Block: eval(), require(), import, any dangerous operations

5. Error Handling

Return null if dependency is missing Return null if division by zero Log errors but don't throw (calculation continues for other KPIs)

INTERFACE:

```
interface ParsedFormula {  type: 'expression' | 'conditional' |
'javascript';  dependencies: string[];  execute: (context: Record<string,
any>) => any;  raw: string; } interface ValidationResult {  isValid:
boolean;  errors: string[];  warnings: string[]; } class FormulaParser {
  parseFormula(formula: string, formulaType: string): ParsedFormula;
  extractDependencies(formula: string): string[];  validateFormula(formula:
string, formulaType: string): ValidationResult;  private
  safeEval(expression: string): number | null;  private
```

```

parseExpression(formula: string, dependencies: string[]): ParsedFormula;
private parseConditional(formula: string, dependencies: string[]):
ParsedFormula; }

```

PROVIDE:

1. Complete FormulaParser class with all methods implemented

2. Unit tests for these cases:

Simple math: "10 + 5" → 15 Variables: "PLANNED_VOL - BASE_VOL" with context

Percentage: "(INCR_VOL / BASE_VOL) * 100" Division by zero: "(100 / 0)" → null

Missing dependency: "PLANNED_VOL - BASE_VOL" with only PLANNED_VOL → null

Conditional: "IF(25 >= 20, 'GREEN', 'RED')" → 'GREEN' Nested conditional: "IF(15 >= 20, 'GREEN', IF(15 >= 10, 'AMBER', 'RED'))" → 'AMBER'

3. Examples of how to use it:

```

const parser = new FormulaParser(); // Example 1: Simple expression
const formula1 = parser.parseFormula("PLANNED_VOL - BASE_VOL", "expression");
const result1 = formula1.execute({ PLANNED_VOL: 4200, BASE_VOL: 3200 });
console.log(result1); // 1000 // Example 2: Conditional
const formula2 = parser.parseFormula("IF(GP_ROI_PCT >= 20, 'GREEN', 'RED')",
"conditional");
const result2 = formula2.execute({ GP_ROI_PCT: 25 });
console.log(result2); // 'GREEN'

```

CONSTRAINTS:

- Pure TypeScript, no external libraries for formula parsing
- Must work in browser (client-side calculation)
- Performance: Parse 100 formulas in < 100ms

--- ## **PROMPT 3: Calculation Dependency Graph**

Create a calculation dependency graph system that determines the correct order to calculate KPIs.

CONTEXT: I have KPIs with formulas that depend on each other:

- INCR_VOL depends on [PLANNED_VOL, BASE_VOL]
- VOL_UPLIFT_PCT depends on [INCR_VOL, BASE_VOL]
- GP_ROI_PCT depends on [INCR_GP, INCR_SPEND]
- INCR_GP depends on [PLANNED_GP, BASE_GP]

Need to calculate in correct order: dependencies first, dependents after.

REQUIREMENTS:

1. Build Dependency Graph

Input: Array of KPI definitions from database Each KPI has: kpi_code, depends_on_kpis (array) Build graph showing what depends on what

2. Topological Sort

Determine calculation order Ensure dependencies are calculated before dependents
Detect circular dependencies (throw error if found)

3. Level Calculation

Assign each KPI a "level" (how many steps from user input) Level 0: User inputs (no dependencies) Level 1: Direct calculations from inputs Level 2: Calculations using Level 1 results, etc.

4. Incremental Updates

When one KPI changes, identify which KPIs need recalculation Return only affected KPIs in dependency order Don't recalculate unaffected KPIs

INTERFACE:

```
interface KPINode {  kpiCode: string;  dependencies: string[]; // KPIs
this depends on  dependents: string[]; // KPIs that depend on this
level: number; // Calculation order level }
class CalculationGraph {
buildGraph(kpis: any[]): Promise<void>;  getCalculationOrder(): string[];
  getAffectedKPIs(changedKPI: string): string[];  getNode(kpiCode:
string): KPINode | undefined;  private calculateLevels(): void;  private
topologicalSort(): string[]; }
```

EXAMPLE USAGE:

```
const graph = new CalculationGraph(); // Sample KPI data
const kpis = [  {
kpi_code: 'PLANNED_VOL', depends_on_kpis: [] },  { kpi_code: 'BASE_VOL',
depends_on_kpis: [] },  { kpi_code: 'INCR_VOL', depends_on_kpis:
['PLANNED_VOL', 'BASE_VOL'] },  { kpi_code: 'VOL_UPLIFT_PCT',
depends_on_kpis: ['INCR_VOL', 'BASE_VOL'] },  { kpi_code: 'BASE_GSV',
depends_on_kpis: ['BASE_VOL', 'BPTT'] },  { kpi_code: 'PLANNED_GSV',
depends_on_kpis: ['PLANNED_VOL', 'BPTT'] },  { kpi_code: 'INCR_GSV',
depends_on_kpis: ['PLANNED_GSV', 'BASE_GSV'] } ];
await graph.buildGraph(kpis); // Get full calculation order
const order = graph.getCalculationOrder();
console.log(order); // Expected:
['PLANNED_VOL', 'BASE_VOL', 'BPTT', 'INCR_VOL', 'BASE_GSV', //
'PLANNED_GSV', 'VOL_UPLIFT_PCT', 'INCR_GSV']
// If PLANNED_VOL changes, what needs recalculation?
const affected = graph.getAffectedKPIs('PLANNED_VOL');
console.log(affected); // Expected:
['INCR_VOL', 'PLANNED_GSV', 'VOL_UPLIFT_PCT', 'INCR_GSV']
// (in correct calculation order)
```

PROVIDE:

1. Complete CalculationGraph class

2. Handle edge cases:

Circular dependency detection: $A \rightarrow B \rightarrow C \rightarrow A$ should throw error Orphaned KPIs: KPIs with no dependencies or dependents Multiple independent branches

3. Unit tests for:

Simple chain: $A \rightarrow B \rightarrow C$ Diamond: $A \rightarrow B, C \rightarrow D$ (both B and C depend on A, D depends on both) Circular: $A \rightarrow B \rightarrow C \rightarrow A$ (should throw error) Incremental update: changing B should only affect B's dependents

4. Console logging for debugging:

Log graph structure when built Log calculation order Log affected KPIs when getting incremental updates

USE:

- TypeScript
- Use the FormulaParser from previous prompt for extracting dependencies
- No external graph libraries (implement yourself)

--- ## **PROMPT 4: Calculation Engine Core**

Create the main calculation engine that executes formulas and manages KPI calculations.

CONTEXT: I have:

1. FormulaParser that can parse and execute formula strings
2. CalculationGraph that determines calculation order
3. Database with KPI definitions

Now need the engine that brings it all together.

REQUIREMENTS:

1. Initialize from Database

Load all active KPIs from Supabase 'kpis' table Build calculation graph Cache parsed formulas for performance

2. Calculate Single SKU

Input: Context with user inputs and master data Execute all formulas in dependency order Return all calculated KPI values

3. Incremental Recalculation

When one value changes, only recalculate affected KPIs Much faster than full recalculation

4. Aggregate to FU Level

Take multiple SKU results Aggregate using method from KPI config (sum, avg, min, max)

5. Format Values for Display

Format based on KPI display_format (number, currency, percentage) Apply decimal_places setting

INTERFACE:

```

interface CalculationContext { // User inputs  planned_volume?: number;
  tactic_values?: Record<string, number>; // e.g., { CPP_ON_PCT: 10 }
// Master data  list_price?: number;  cogs_per_piece?: number;
base_volume?: number; // Pre-calculated values  calculated?:
Record<string, any>; } interface CalculationResult {  kpiCode: string;
value: any;  formatted: string;  error?: string; } class
CalculationEngine {  initialize(): Promise<void>;  calculateSKU(context:
CalculationContext): Record<string, CalculationResult>;  recalculate(
changedKPI: string,    newValue: any,    currentContext: Record<string,
any>  ): Record<string, CalculationResult>;  aggregateFU(skuResults:
Record<string, any>[]): Record<string, any>;  private formatValue(value:
any, kpi: any): string; }

```

DEPENDENCIES: Use these from previous prompts:

```

import { formulaParser } from './formulaParser'; import { calculationGraph
} from './calculationGraph'; import { supabase } from './supabaseClient';

```

EXAMPLE USAGE:

```

const engine = new CalculationEngine(); await engine.initialize(); //
Calculate all KPIs for one SKU const context = {  planned_volume: 4200,
base_volume: 3200,  list_price: 4.00,  cogs_per_piece: 1.80,
tactic_values: {    CPP_ON_PCT: 10,    CPP_OFF_PCT: 5  } }; const
results = engine.calculateSKU(context); console.log(results); /* Expected
output: {  INCR_VOL: { kpiCode: 'INCR_VOL', value: 1000, formatted:
'1,000' },  VOL_UPLIFT_PCT: { kpiCode: 'VOL_UPLIFT_PCT', value: 31.25,
formatted: '31.25%' },  BASE_GSV: { kpiCode: 'BASE_GSV', value: 12800,
formatted: '$12,800.00' },  PLANNED_GSV: { kpiCode: 'PLANNED_GSV', value:
16800, formatted: '$16,800.00' },  ... all other calculated KPIs } */ //
Incremental update when planned volume changes const updatedResults =
engine.recalculate(  'PLANNED_VOL',  5000, // new value  results //
current state ); console.log(updatedResults); // Only contains recalculated
KPIs that depend on PLANNED_VOL // Aggregate multiple SKUs to FU level
const sku1Results = { INCR_VOL: { value: 1000 }, PLANNED_GP: { value: 2500
} }; const sku2Results = { INCR_VOL: { value: 750 }, PLANNED_GP: { value:
1800 } }; const sku3Results = { INCR_VOL: { value: 500 }, PLANNED_GP: {
value: 1200 } }; const fuResults = engine.aggregateFU([sku1Results,
sku2Results, sku3Results]); console.log(fuResults); /* Expected output: {
INCR_VOL: 2250, // sum  PLANNED_GP: 5500 // sum } */

```

PROVIDE:

2. Error handling:

Handle missing dependencies gracefully (return null, don't crash) Handle division by zero Log calculation errors but continue with other KPIs

3. Performance optimization:

Cache parsed formulas (don't re-parse on every calculation) Cache KPI definitions Only recalculate affected KPIs

4. Formatting examples:

Currency: 12500.50 → "\$12,500.50" Percentage: 25.5 → "25.50%" (respect decimal_places) Number: 1000 → "1,000" (thousand separators)

5. Integration with Supabase:

```
async initialize() {  const { data: kpis, error } = await supabase
.from('kpis')      .select('*')      .eq('is_active', true)
.order('calculation_order');  // Build graph and cache }
```

USE:

- TypeScript with strict types
- FormulaParser and CalculationGraph from previous prompts
- Supabase client
- console.log for debugging key steps

--- ## **PROMPT 5: React Hook for Plan Calculations**

Create a React hook that manages plan calculations with real-time updates.

CONTEXT: I have a calculation engine that can calculate KPIs. Now need React integration for the planning grid where:

- Planners enter planned volumes at SKU level
- Planners enter tactic values (discounts, fees) at FU level
- All KPIs recalculate automatically
- Results aggregate to FU and Plan levels
- Auto-save to Supabase after 2 seconds of no changes

REQUIREMENTS:

1. State Management

Store FUs (each containing multiple SKUs) Store calculated values for each SKU Store calculated values for each FU (aggregated) Store grand totals (plan level aggregated)

2. Update Functions

updateSKUVolume(fuld, skuld, newVolume) - triggers SKU recalculation

updateFUTactic(fuld, tacticCode, newValue) - triggers all SKU recalculations in that FU

3. Automatic Aggregation

When SKU values change → aggregate to FU level When FU values change → aggregate to Plan level

4. Debounced Auto-Save

Save to Supabase 2 seconds after last change Save to plan_sku_volumes and plan_mechanic_values tables

5. Real-Time Feedback

isCalculating flag to show loading state Optimistic UI updates (update state immediately, save async)

INTERFACE:

```
interface SKUData {  sku_id: string;  sku_code: string;  sku_name:
string;  planned_volume: number;  base_volume: number;  list_price:
number;  cogs_per_piece: number;  calculated: Record<string, any>; // All
calculated KPI values } interface FUData {  fu_id: string;  fu_code:
string;  fu_name: string;  tactics: Record<string, number>; // e.g., {
CPP_ON_PCT: 10, VISIBILITY_MTPH: 2000 }  skus: SKUData[];  calculated:
Record<string, any>; // Aggregated from SKUs } function
usePlanCalculations(planId: string) {  // Returns:  return {    fus:
FUData[],    grandTotals: Record<string, any>,    isCalculating: boolean,
    updateSKUVolume: (fuId: string, skuId: string, newVolume: number) =>
void,    updateFUTactic: (fuId: string, tacticCode: string, newValue:
number) => void,    addFU: (fuId: string) => Promise<void>,    removeFU:
(fuId: string) => void  }; }
```

DEPENDENCIES:

```
import { useState, useEffect, useCallback } from 'react'; import {
calculationEngine } from '../services/calculationEngine'; import { supabase
} from '../lib/supabaseClient'; import { debounce } from 'lodash'; // or
implement your own
```

EXAMPLE USAGE IN COMPONENT:

```
function PlanningGrid({ planId }) {  const {    fus,    grandTotals,
isCalculating,    updateSKUVolume,    updateFUTactic  } =
usePlanCalculations(planId);  return (    <div>      {/* Grand Totals
*/}      <div>        <h3>Plan Total GP ROI: {grandTotals.GP_ROI_PCT}%
</h3>        <h3>Plan Total Spend: ${grandTotals.TOTAL_PLANNED_SPEND}</h3>
      </div>      {/* Grid */}      {fus.map(fu => (        <div key=
{fu.fu_id}>          <h4>{fu.fu_name}</h4>          {/* FU-
level tactic inputs */}          <input              type="number"
              value={fu.tactics.CPP_ON_PCT || 0}              onChange={
(e) =>
updateFUTactic(fu.fu_id, 'CPP_ON_PCT', parseFloat(e.target.value))}
        </div>      )
    )}
```

```

    <span>{sku.sku_name}</span>                                { /* SKU-level
volume input */}      <input                                type="number"
    value={sku.planned_volume}                                onChange={ (e) =>
updateSKUVolume(fu.fu_id, sku.sku_id, parseFloat(e.target.value))}
    placeholder="Planned Volume"                                />                                { /*
Calculated values */}      <span>iVol: {sku.calculated.INCR_VOL}
</span>                                <span>GP ROI: {sku.calculated.GP_ROI_PCT}%</span>
    </div>                                )}}      </div>                                )}}      {isCalculating
&& <div>Calculating...</div>}      </div>      ); }

```

PROVIDE:

1. Complete usePlanCalculations hook

2. Load initial data from Supabase:

// On mount, load: - Plan FUs from plan_fus table - SKUs for each FU from fus and fu_skus tables - Master data (list_price, cogs) from skus table - Historical base volumes from plan_sku_volumes or historical_volumes table - Current planned volumes and tactics from plan_sku_volumes and plan_mechanic_values

3. Update flow diagram in comments:

// Update Flow: // 1. User changes SKU planned volume // 2. updateSKUVolume called // 3. Calculate new SKU KPIs using engine.recalculate() // 4. Update SKU state with new calculated values // 5. Aggregate all SKUs to FU level // 6. Aggregate all FUs to Plan level // 7. Update grandTotals state // 8. Trigger debounced save after 2 seconds

4. Save function:

```

async function savePlanToDatabase(fusData: FUData[]): Promise<void> { //
Batch upsert to: // - plan_sku_volumes (sku_id, planned_volume,
calculated_values JSONB) // - plan_mechanic_values (plan_fu_id,
mechanic_code, value) }

```

5. Error handling: Show toast notification on save error Retry failed saves Don't block UI if save fails

USE:

- React 18+ with hooks
- TypeScript
- CalculationEngine from previous prompt
- Supabase client
- lodash debounce (or custom implementation)

--- ## **PROMPT 6: Dynamic Column Configuration Hook**

Create a React hook that dynamically determines which columns to show in the planning grid based on plan context.

CONTEXT: The planning grid should only show:

1. KPIs that are configured to display (`show_in_grid = true`)
2. Tactics that apply to the plan's channel and category

Example:

- Plan is for "NKA" channel and "Dairy" category
- Only show tactics where `applicable_channels` includes "NKA" OR "ALL"
- Only show tactics where `applicable_categories` includes "Dairy" OR "ALL"

REQUIREMENTS:

1. Fetch KPI Columns

Load from 'kpis' table where `show_in_grid = true` Order by `calculation_order` or `custom_display_order` Return column configuration for grid

2. Fetch Tactic Columns

Load from 'promo_mechanics' table Filter by plan context (channel, category) Only show applicable tactics

3. Group Columns

Group by `kpi_group` (Volume, Profit, ROI, etc.) Provide grouped structure for headers

4. Column Metadata

Include display name, format, width, tooltip

INTERFACE:

```
interface KPIColumn {  kpiCode: string;  kpiName: string;  displayName: string;  kpiGroup: string;  displayFormat: 'number' | 'currency' | 'percentage';  decimalPlaces: number;  width: number; // pixels  formula: string; // for tooltip  description: string;  isEditable: boolean; // false for calculated, true for user input  editableAt: 'sku' | 'fu' | null; // where it can be edited } interface TacticColumn {  code: string;  name: string;  inputType: 'percentage' | 'currency';  minValue: number;  maxValue: number;  width: number;  description: string;  category: string; // 'on_invoice_discount', 'off_invoice_discount', etc. } interface ColumnGroups {  [groupName: string]: KPIColumn[]; } function useKPIColumns(planContext: { channel: string; category: string }) {  return {    columns: KPIColumn[],    tacticColumns: TacticColumn[],    columnGroups: ColumnGroups,    isLoading: boolean,    error: Error | null  }; }
```

EXAMPLE USAGE:

```
function PlanningGrid({ planId, planContext }) {  const {    columns,    tacticColumns,    columnGroups,    isLoading  } =
```

```

useKPIColumns(planContext); if (isLoading) return <div>Loading columns...
</div>; return (
  <table>
    <thead>
      <tr>
        <th>Item Name</th>
        <th>Base Vol</th>
        <th>Planned Vol</th>
        <th>Dynamic KPI columns */>
        {columns.map(col => (
          <th key={col.kpiCode} style={{ width:
            col.width }}>
            {col.displayName}
            <Tooltip text=
            {col.description} />
            </th>
          )))
        <th>Dynamic
        tactic columns */>
        {tacticColumns.map(tactic => (
          <th
            key={tactic.code} style={{ width: tactic.width }}>
            {tactic.name}
            ({tactic.inputType === 'percentage' ? '%' :
            '$'})
            </th>
          )))
        <th>GP ROI %</th>
        <th>RAG</th>
        </tr>
      </thead>
      <tbody>
        <tr>
          <td colspan={columns.length}>
            <div>Grid rows */>
          </td>
        </tr>
      </tbody>
    </table>
  ); }
// Usage with context <PlanningGrid planId="plan-123
" planContext={{ channel: 'NKA', category: 'Dairy' }} />
// Will only show tactics applicable to NKA channel and Dairy category

```

FILTERING LOGIC EXAMPLE:

```

// Plan context const planContext = { channel: 'NKA', category: 'Dairy' };
// Tactic in database const tactic1 = { code: 'CPP_ON_PCT', name: 'CPP
On-Invoice %', applicable_channels: ['NKA', 'Traditional Trade', 'E-
Commerce'], applicable_categories: ['ALL'] }; // Result: SHOW (NKA is in
applicable_channels) const tactic2 = { code: 'WS_TPR_ON_PCT', name: 'WS
TPR On-Invoice %', applicable_channels: ['Wholesale'],
applicable_categories: ['ALL'] }; // Result: HIDE (Wholesale not in
applicable_channels, plan is NKA) const tactic3 = { code:
'VISIBILITY_MTPH', name: 'Visibility Lumpsum MT/PH',
applicable_channels: ['NKA'], applicable_categories: ['Dairy',
'Beverages'] }; // Result: SHOW (Both NKA and Dairy match) const tactic4 =
{ code: 'EXPORT_FEE', name: 'Export Handling Fee',
applicable_channels: ['Export'], applicable_categories: ['ALL'] }; //
Result: HIDE (Export not in applicable_channels)

```

PROVIDE:

1. Complete useKPIColumns hook with:

Supabase queries to fetch KPIs and tactics
 Filtering logic for tactics based on plan
 context
 Grouping logic for column groups
 Loading and error states

2. Supabase query examples:

```

.order('calculation_order'); // Fetch tactic columns with filtering const {
data: tactics } = await supabase .from('promo_mechanics') .select('*')
.eq('is_active', true) .eq('show_in_grid', true); // Then filter in
code: const filteredTactics = tactics.filter(tactic => { const
channelMatch = tactic.applicable_channels.includes('ALL') ||
tactic.applicable_channels.includes(planContext.channel); const
categoryMatch = tactic.applicable_categories.includes('ALL') ||
tactic.applicable_categories.includes(planContext.category); return
channelMatch && categoryMatch; });

```

3. Default column configuration:

```

// Always visible columns (cannot be hidden) const FIXED_COLUMNS = [ {
code: 'item_name', name: 'Item Name', width: 250, fixed: 'left' }, {
code: 'base_vol', name: 'Base Vol', width: 100 }, { code: 'planned_vol',
name: 'Planned Vol', width: 120, editable: true }, ]; const
FIXED_RIGHT_COLUMNS = [ { code: 'gp_roi_pct', name: 'GP ROI %', width:
120, fixed: 'right' }, { code: 'rag_status', name: 'RAG', width: 100,
fixed: 'right' } ];

```

4. Column grouping for headers:

```

// Return structure like: const columnGroups = { 'Volume': [ {
kpiCode: 'BASE_VOL', displayName: 'Base Vol', ... }, { kpiCode:
'PLANNED_VOL', displayName: 'Planned Vol', ... }, { kpiCode:
'INCR_VOL', displayName: 'iVol', ... } ], 'Profit': [ { kpiCode:
'BASE_GP', displayName: 'Base GP', ... }, { kpiCode: 'PLANNED_GP',
displayName: 'Planned GP', ... } ], 'ROI': [ { kpiCode:
'GP_ROI_PCT', displayName: 'GP ROI %', ... } ] };

```

5. User preference support (bonus):

```

// Save user's column visibility preferences const saveColumnPreferences =
async (userId: string, preferences: any) => { await supabase
.from('user_preferences') .upsert({ user_id: userId,
preference_type: 'column_visibility', preferences: preferences
}); }; // Load and apply preferences const loadColumnPreferences = async
(userId: string) => { const { data } = await supabase
.from('user_preferences') .select('preferences') .eq('user_id',
userId) .eq('preference_type', 'column_visibility') .single();
return data?.preferences || {}; };

```

USE:

- React hooks (useState, useEffect)

- Supabase client
- Handle loading states with skeleton UI

--- ## **PROMPT 7: Planning Grid Components**

Create the main React components for the planning grid with dynamic columns and real-time calculations.

CONTEXT: I have:

- usePlanCalculations hook (manages data and calculations)
- useKPIColumns hook (provides column configuration)
- Need to build the actual grid UI with FU/SKU hierarchy

REQUIREMENTS:

- 1. PlanningGrid Component** - Main container
Sticky header with column names Horizontal and vertical scroll Fixed left columns (item name) Fixed right columns (ROI, RAG)
- 2. FURow Component** - Parent row (blue background)
Expand/collapse icon Shows aggregated values (read-only) Tactic inputs (editable at FU level) Bold styling
- 3. SKURow Component** - Child row (white background)
Indented with arrow (↳) Volume input (editable at SKU level) Tactic values inherited from FU (read-only, gray) Calculated KPI values
- 4. EditableCell Component** - Inline editing
Click to edit Real-time validation Auto-save on blur Keyboard shortcuts (Enter, Escape, Tab)
- 5. DynamicGridHeader** - Column headers
Group headers (Volume, Profit, ROI) Individual column headers Tooltips with formula info Resize columns

COMPONENT STRUCTURE:

```
// PlanningGrid.tsx
interface PlanningGridProps {
  planId: string;
  planContext: { channel: string; category: string };
}
export function PlanningGrid({ planId, planContext }: PlanningGridProps) {
  const { fus, grandTotals, isCalculating, updateSKUVolume, updateFUTactic } = usePlanCalculations(planId);
  const { columns, tacticColumns } = useKPIColumns(planContext);
  const [expandedFUs, setExpandedFUs] = useState<Set<string>>(new Set());
  return (
    <div className="planning-grid-container">
      {/* Grand Totals Panel - separate component */}
      <GrandTotalsPanel totals={grandTotals} />
      <div className="toolbar">
        <button>+ Add FU</button>
        <button>Expand All</button>
        <button>Collapse All</button>
        <button>Export</button>
        <button onClick=
```

```

{saveManually}>Save</button>      </div>      {/* Grid */}      <div
className="grid-scroll-container">      <table className="planning-grid-
table">      <DynamicGridHeader      columns={columns}
      tacticColumns={tacticColumns}      />      <tbody>
      {fus.map(fu => (      <React.Fragment key={fu.fu_id}>
        <FURow      fu={fu}      columns={columns}
          tacticColumns={tacticColumns}
isExpanded={expandedFUs.has(fu.fu_id)}      onToggle={() =>
toggleFU(fu.fu_id)}      onTacticChange={(tacticCode, value)
=>      updateFUTactic(fu.fu_id, tacticCode, value)
      }      />      {expandedFUs.has(fu.fu_id)
      && fu.skus.map(sku => (      <SKURow      key=
{sku.sku_id}      sku={sku}      fuTactics=
{fu.tactics}      columns={columns}
tacticColumns={tacticColumns}      onVolumeChange={(value)
=>      updateSKUVolume(fu.fu_id, sku.sku_id, value)
      }      />      )))}
</React.Fragment>      )))}      </tbody>      </table>
</div>      {isCalculating && (      <div className="calculating-
overlay">      <Spinner />      Calculating...      </div>
      )}      </div>      ); }
// FURow.tsx interface FURowProps { fu: FUData; columns: KPIColumn[];
tacticColumns: TacticColumn[]; isExpanded: boolean; onToggle: () =>
void; onTacticChange: (tacticCode: string, value: number) => void; }
export function FURow({ fu, columns, tacticColumns, isExpanded,
onToggle, onTacticChange }: FURowProps) { return ( <tr
className="fu-row bg-blue-50 font-bold">      {/* Expand/Collapse + Name
*/}      <td className="sticky left-0 bg-blue-50">      <button
onClick={onToggle}>      {isExpanded ? '−' : '+' }      </button>
      <span className="ml-2">{fu.fu_code} - {fu.fu_name}</span>
</td>      {/* Master Data (read-only, from SKUs) */}      <td
className="text-gray-400">-</td>      <td className="text-gray-400">-</td>
      {/* Volume columns (aggregated, read-only) */}      <td
className="text-right">{formatNumber(fu.calculated.BASE_VOL)}</td>
<td className="text-right">{formatNumber(fu.calculated.PLANNED_VOL)}</td>
      {/* Dynamic KPI columns (calculated) */}      {columns.map(col => (
      <td key={col.kpiCode} className="text-right">

```

```

        {col.displayFormat}                decimals={col.decimalPlaces}                />
        </td>                )))      { /* Tactics columns (EDITABLE at FU level) */}
        {tacticColumns.map(tactic => (      <td key={tactic.code}
className="bg-white">                <EditableCell                value=
{fu.tactics[tactic.code] || 0}                type={tactic.inputType}
        min={tactic.minValue}                max={tactic.maxValue}
onChange={value => onTacticChange(tactic.code, value)}                />
        </td>                )))      { /* ROI with RAG */}      <td className="sticky
right-20 bg-blue-50 text-right">                <RAGCell                value=
{fu.calculated.GP_ROI_PCT}                ragStatus={fu.calculated.RAG_STATUS}
        />                </td>      <td className="sticky right-0 bg-blue-50">
        <RAGBadge status={fu.calculated.RAG_STATUS} />                </td>      </tr>      );
}

// SKURow.tsx interface SKURowProps {  sku: SKUData;  fuTactics:
Record<string, number>;  columns: KPIColumn[];  tacticColumns:
TacticColumn[];  onVolumeChange: (value: number) => void; } export
function SKURow({  sku,  fuTactics,  columns,  tacticColumns,
onVolumeChange }: SKURowProps) {  return (    <tr className="sku-row
hover:bg-gray-50">      { /* Indented name */}      <td className="sticky
left-0 bg-white pl-10">        <span className="text-gray-600">↳
{sku.sku_code} - {sku.sku_name}</span>        </td>      { /* Master Data
*/}      <td className="text-right">{formatCurrency(sku.list_price)}</td>
        <td className="text-right">{formatCurrency(sku.cogs_per_piece)}</td>
        { /* Volume */}      <td className="text-right text-gray-600">
{formatNumber(sku.base_volume)}</td>      <td className="bg-yellow-50">
        { /* EDITABLE volume input */}      <EditableCell                value=
{sku.planned_volume}                type="number"                min={0}
onChange={onVolumeChange}                />      </td>      { /* Dynamic KPI
columns (calculated, animated) */}      {columns.map(col => (      <td
key={col.kpiCode} className="text-right">                <AnimatedValue
        value={sku.calculated[col.kpiCode]}                format=
{col.displayFormat}                decimals={col.decimalPlaces}
highlightChange={true}                />      </td>      )))      { /*
Tactics columns (READ-ONLY, inherited from FU) */}
        {tacticColumns.map(tactic => (      <td key={tactic.code}
className="text-right text-gray-500 bg-gray-50">

```

```

<span className="text-xs ml-1">(from FU)</span>                                </>                                )
: '-'}                                </td>                                )}}                                { /* ROI with RAG */}                                <td
className="sticky right-20 bg-white text-right">                                <RAGCell
value={sku.calculated.GP_ROI_PCT}                                ragStatus=
{sku.calculated.RAG_STATUS}                                />                                </td>                                <td
className="sticky right-0 bg-white">                                <RAGBadge status=
{sku.calculated.RAG_STATUS} />                                </td>                                </tr>                                ); }
// EditableCell.tsx interface EditableCellProps { value: number; type:
'number' | 'percentage' | 'currency'; min?: number; max?: number;
onChange: (value: number) => void; } export function EditableCell({
value, type, min, max, onChange }: EditableCellProps) { const
[localValue, setLocalValue] = useState(String(value)); const [isEditing,
setIsEditing] = useState(false); const [error, setError] =
useState<string | null>(null); const inputRef = useRef<HTMLInputElement>
(null); // Update local value when prop changes (if not editing)
useEffect(() => { if (!isEditing) { setLocalValue(String(value));
    } }, [value, isEditing]); const handleFocus = () => {
setIsEditing(true); setTimeout(() => inputRef.current?.select(), 0);
}; const handleBlur = () => { setIsEditing(false);
validateAndSave(); }; const handleKeyDown = (e: React.KeyboardEvent) =>
{ if (e.key === 'Enter') { validateAndSave();
inputRef.current?.blur(); } else if (e.key === 'Escape') {
setLocalValue(String(value)); setError(null);
inputRef.current?.blur(); } }; const validateAndSave = () => {
const numValue = parseFloat(localValue); if (isNaN(numValue)) {
setError('Invalid number'); return; } if (min !== undefined
&& numValue < min) { setError(`Minimum: ${min}`); return; }
if (max !== undefined && numValue > max) { setError(`Maximum:
${max}`); return; } setError(null); onChange(numValue);
}; const displayValue = isEditing ? localValue : type ===
'percentage' ? `${value}%` : type === 'currency' ?
`$${value.toLocaleString()}` : value.toLocaleString(); return (
<div className="relative"> <input ref={inputRef}
type="text" value={displayValue} onChange={(e) =>
setLocalValue(e.target.value)} onFocus={handleFocus}
onBlur={handleBlur} onKeyDown={handleKeyDown} className={`

```

```
'border-red-500 bg-red-50' : 'border-gray-300'} `} />
{error && ( <div className="absolute top-full left-0 mt-1 text-xs
text-red-600 z-10 bg-white border border-red-300 px-2 py-1 rounded shadow">
  △ {error} </div> )} </div> ); }
```

PROVIDE:

1. All component files:

PlanningGrid.tsx (main container) DynamicGridHeader.tsx FURow.tsx SKURow.tsx
EditableCell.tsx AnimatedValue.tsx RAGCell.tsx RAGBadge.tsx FormattedValue.tsx

2. CSS/Tailwind classes for:

Sticky columns (left and right) Horizontal scroll Zebra striping Hover effects FU row
styling (blue background) Editable cell highlighting

3. Keyboard navigation:

Tab to move between editable cells Arrow keys to navigate grid Enter to edit, Esc to
cancel

4. Helper functions:

```
function formatNumber(value: number): string; function
formatCurrency(value: number): string; function formatPercentage(value:
number): string; function formatTacticValue(value: number, type: string):
string;
```

5. Responsive design: Desktop: Full grid Tablet: Horizontal scroll Mobile: Card-based layout (simplified view)

USE:

- React 18+ with TypeScript
- Tailwind CSS for styling
- Hooks from previous prompts
- shadcn/ui components (optional, or plain HTML)

--- ## **PROMPT 8: Grand Totals Panel & Animated Values**

Create the Grand Totals Panel component and AnimatedValue component for real-time
visual feedback.

CONTEXT: Above the planning grid, need a sticky panel showing plan-level aggregated
KPIs that updates in real-time as users edit values.

REQUIREMENTS:

1. Grand Totals Panel

Sticky at top (always visible) 6 metric cards in a row Real-time updates with smooth
animations Color-coded based on performance

2. Metric Cards:

Volume: Total planned volume, incremental volume, uplift % Turnover: Total planned turnover, incremental turnover Spend: Total spend, on-invoice vs off-invoice breakdown Profit: Incremental GP, base GP, planned GP Efficiency: GP ROI %, GP Margin % Status: Overall RAG status, item breakdown

3. AnimatedValue Component

Count-up animation when value changes Highlight in green if increased, red if decreased Smooth transitions

4. RAG Visualization

Large colored badge Bar showing distribution (% green, amber, red items) Budget impact indicator

COMPONENTS:

```
// GrandTotalsPanel.tsx interface GrandTotalsPanelProps {  totals:
Record<string, any>; } export function GrandTotalsPanel({ totals }:
GrandTotalsPanelProps) {  return (    <div className="sticky top-0 z-10
bg-white border-b-4 border-blue-500 shadow-lg p-4">      <h2
className="text-lg font-bold mb-3">PLAN PERFORMANCE OVERVIEW</h2>
      <div className="grid grid-cols-6 gap-4">        {/* Volume Card */}
        <MetricCard          icon="📦 "          title="VOLUME"
mainValue={totals.PLANNED_VOL}          mainLabel="Planned Total"
mainFormat="number"          mainUnit="pcs"          secondaryValue=
{totals.INCR_VOL}          secondaryLabel="Incremental"
tertiaryValue={totals.VOL_UPLIFT_PCT}          tertiaryLabel="vs base"
tertiaryFormat="percentage"          />        {/* Turnover Card */}
        <MetricCard          icon="💰 "          title="TURNOVER"
mainValue={totals.PLANNED_TO}          mainLabel="Planned Total"
mainFormat="currency"          secondaryValue={totals.INCR_TO}
secondaryLabel="Incremental"          tertiaryValue={totals.TO_UPLIFT_PCT}
tertiaryLabel="vs base"          tertiaryFormat="percentage"
        />        {/* Spend Card */}
        <MetricCard          icon="💵 "          title="SPEND"
mainValue={totals.TOTAL_PLANNED_SPEND}
mainLabel="Total Planned"          mainFormat="currency"
secondaryValue={totals.INCR_SPEND}          secondaryLabel="Incremental"
breakdown={{          'On-Invoice':
totals.TOTAL_ON_INVOICE_SPEND,          'Off-Invoice':
totals.TOTAL_OFF_INVOICE_SPEND          }}          />        {/* Profit
```

```

        mainFormat="currency"                secondaryValue={totals.BASE_GP}
        secondaryLabel="Base GP"              tertiaryValue={totals.PLANNED_GP}
        tertiaryLabel="Planned GP"            />          { /* Efficiency Card
*/}      <MetricCard icon="🇩🇪" title="EFFICIENCY"
        mainValue={totals.GP_ROI_PCT}         mainLabel="GP ROI"
        mainFormat="percentage"               isHighlight={true}          ragStatus=
{totals.RAG_STATUS}          secondaryValue={totals.PLANNED_GM_PCT}
        secondaryLabel="GP Margin"            secondaryFormat="percentage"
    />          { /* Overall Status Card */}      <StatusCard
ragStatus={totals.RAG_STATUS}          ragBreakdown={totals.RAG_BREAKDOWN}
// { green: 16, amber: 3, red: 1 }          budgetImpact=
{totals.BUDGET_IMPACT_PCT}          />          </div>          </div>    ); }
// MetricCard.tsx interface MetricCardProps { icon: string; title:
string; mainValue: number; mainLabel: string; mainFormat: 'number' |
'currency' | 'percentage'; mainUnit?: string; secondaryValue?: number;
    secondaryLabel?: string; secondaryFormat?: string; tertiaryValue?:
number; tertiaryLabel?: string; tertiaryFormat?: string;
isHighlight?: boolean; ragStatus?: string; breakdown?: Record<string,
number>; } export function MetricCard({ icon, title, mainValue,
mainLabel, mainFormat, mainUnit, secondaryValue, secondaryLabel,
tertiaryValue, tertiaryLabel, tertiaryFormat, isHighlight,
ragStatus, breakdown }: MetricCardProps) { const bgColor = isHighlight
&& ragStatus ? ragStatus === 'GREEN' ? 'bg-green-100' : ragStatus
=== 'AMBER' ? 'bg-yellow-100' : 'bg-red-100' : 'bg-gray-50';
return (      <div className={`border rounded-lg p-3 ${bgColor}`}>          { /*
Header */}      <div className="flex items-center gap-2 mb-2">
<span className="text-2xl">{icon}</span>          <span className="font-bold
text-sm text-gray-700">{title}</span>          </div>          { /* Main Value */}
        <div className="mb-2">          <div className="text-xs text-gray-
600">{mainLabel}</div>          <div className="text-2xl font-bold">
        <AnimatedValue          value={mainValue}          format=
{mainFormat}          highlightChange={true}          />
{mainUnit && <span className="text-sm ml-1">{mainUnit}</span>}
</div>          </div>          { /* Secondary Value */}          {secondaryValue !==
undefined && (          <div className="text-sm mb-1">          <span
className="text-gray-600">{secondaryLabel}: </span>          <span

```

```

    }}      {/* Tertiary Value */}      {tertiaryValue !== undefined && (
      <div className="text-sm">          <span className="text-gray-
600">{tertiaryLabel}: </span>          <span className="font-semibold">
      <AnimatedValue                    value={tertiaryValue}
format={tertiaryFormat || mainFormat}      />          </span>
      </div>      )}      {/* Breakdown */}      {breakdown && (
<div className="mt-2 pt-2 border-t text-xs space-y-1">
  {Object.entries(breakdown).map(([key, value]) => (          <div key=
{key} className="flex justify-between">          <span
className="text-gray-600">{key}:</span>          <span
className="font-semibold">          <AnimatedValue value={value}
format="currency" decimals={0} />          </span>          </div>
    )}}      </div>      )}      </div>    ); }
// AnimatedValue.tsx interface AnimatedValueProps {  value: number | null;
  format: 'number' | 'currency' | 'percentage';  decimals?: number;
highlightChange?: boolean; } export function AnimatedValue({  value,
format,  decimals = 2,  highlightChange = false }: AnimatedValueProps) {
  const [displayValue, setDisplayValue] = useState(value);  const
[isAnimating, setIsAnimating] = useState(false);  const prevValueRef =
useRef(value);  useEffect(() => {    if (value !== prevValueRef.current
&& highlightChange && value !== null) {      setIsAnimating(true);
      // Count-up animation      const start = prevValueRef.current || 0;
      const end = value;      const duration = 300; // ms      const steps
= 20;      const increment = (end - start) / steps;      let currentStep
= 0;      const interval = setInterval(() => {        currentStep++;
      if (currentStep >= steps) {          setDisplayValue(end);
clearInterval(interval);          setTimeout(() => setIsAnimating(false),
500);        } else {          setDisplayValue(start + increment *
currentStep);        }      }, duration / steps);
      prevValueRef.current = value;      return () => clearInterval(interval);
    } else {      setDisplayValue(value);      prevValueRef.current =
value;    }  }, [value, highlightChange]);  if (displayValue === null ||
displayValue === undefined) {    return <span className="text-gray-400">-
</span>;  }  const formatted = formatValue(displayValue, format,
decimals);  const changeDirection =    (value || 0) >
(prevValueRef.current || 0) ? 'up' : 'down';  return (    <span

```

```

px-1 rounded'                : 'bg-red-200 text-red-800 px-1 rounded'
: ''                }      `}      >      {formatted}      </span>      ); }
function formatValue( value: number, format: string, decimals: number
): string { switch (format) { case 'currency': return new
Intl.NumberFormat('en-US', { style: 'currency', currency:
'USD', minimumFractionDigits: decimals,
maximumFractionDigits: decimals }).format(value); case
'percentage': return `${value.toFixed(decimals)}%`; case
'number': default: return new Intl.NumberFormat('en-US', {
minimumFractionDigits: decimals, maximumFractionDigits: decimals
}).format(value); } }
// StatusCard.tsx interface StatusCardProps { ragStatus: string;
ragBreakdown: { green: number; amber: number; red: number };
budgetImpact: number; } export function StatusCard({ ragStatus,
ragBreakdown, budgetImpact }: StatusCardProps) { const total =
ragBreakdown.green + ragBreakdown.amber + ragBreakdown.red; const
greenPct = (ragBreakdown.green / total) * 100; const amberPct =
(ragBreakdown.amber / total) * 100; const redPct = (ragBreakdown.red /
total) * 100; return ( <div className="border rounded-lg p-3 bg-
white"> <div className="text-sm font-bold text-gray-700 mb-3">OVERALL
STATUS
</div>
{/* Large RAG Badge */}
{ragStatus === 'GREEN' ? '🟢' : ragStatus === 'AMBER' ? '🟡' : '🔴' } {ragStatus}
{ragStatus === 'GREEN' ? 'Excellent Performance' : ragStatus === 'AMBER' ?
'Acceptable Performance' : 'Needs Attention'}
{/* Item Breakdown Bar */}
Item Breakdown:
{ragBreakdown.green > 0 && ragBreakdown.green}
{ragBreakdown.amber > 0 && ragBreakdown.amber}
{ragBreakdown.red > 0 && ragBreakdown.red}
✓ {ragBreakdown.green} Green ⚡ {ragBreakdown.amber} Amber ⚠️
{ragBreakdown.red} Red
{/* Budget Impact */}
Budget Impact: {budgetImpact.toFixed(1)}%

{/* Action Buttons */}

```

```
);}
```

PROVIDE:

1. All component files:

GrandTotalsPanel.tsx MetricCard.tsx AnimatedValue.tsx StatusCard.tsx

2. Animation logic:

Smooth count-up animation (300ms) Highlight changes with colored background

Fade highlight after 500ms

3. Responsive design:

Desktop: 6 cards in a row Tablet: 3 cards in 2 rows Mobile: 1-2 cards per row, vertical scroll

4. Calculation of totals object in parent:

```
// In usePlanCalculations hook const grandTotals = useMemo(() => { //
Aggregate all FU calculated values const allFUValues = fus.map(fu =>
fu.calculated); const aggregated =
calculationEngine.aggregateFU(allFUValues); // Calculate RAG breakdown
let greenCount = 0, amberCount = 0, redCount = 0; fus.forEach(fu => {
fu.skus.forEach(sku => { const ragStatus =
sku.calculated.RAG_STATUS; if (ragStatus === 'GREEN') greenCount++;
else if (ragStatus === 'AMBER') amberCount++; else if (ragStatus
=== 'RED') redCount++; }); }); return { ...aggregated,
RAG_BREAKDOWN: { green: greenCount, amber: amberCount, red: redCount },
BUDGET_IMPACT_PCT: (aggregated.TOTAL_PLANNED_SPEND / monthlyBudget) * 100
}; }, [fus, monthlyBudget]);
```

5. Unit tests for AnimatedValue: Test count-up from 100 to 200 Test count-down from 200 to 100 Test null values Test rapid changes (should cancel previous animation)

USE:

- React 18+ with hooks
- Tailwind CSS
- TypeScript
- Smooth CSS transitions

--- ## **PROMPT 9: RAG Status Components & Visualization**

Create RAG (Red-Amber-Green) status visualization components for the planning grid.

CONTEXT: Each SKU and FU has a RAG status based on GP ROI % thresholds configured by admins. Need to display this visually in the grid with colors, badges, and tooltips.

REQUIREMENTS:

- 1. RAGCell Component** - Shows ROI value with colored background
- 2. RAGBadge Component** - Shows colored status badge with label
- 3. RAGTooltip Component** - Detailed breakdown on hover

4. RAG Threshold Indicator - Visual range showing where current value falls

COMPONENTS:

```
// RAGCell.tsx interface RAGCellProps { value: number | null;
ragStatus: 'GREEN' | 'AMBER' | 'RED' | 'GRAY'; showTooltip?: boolean;
thresholds?: { green: number; amber: number; red: number; };
} export function RAGCell({ value, ragStatus, showTooltip = true,
thresholds = { green: 20, amber: 10, red: 10 } }: RAGCellProps) { const
[showDetail, setShowDetail] = useState(false); if (value === null ||
value === undefined) { return ( <div className="text-gray-400
text-center"> - </div> ); } // Color based on RAG
status const bgColor = ragStatus === 'GREEN' ? 'bg-green-100 text-
green-800' : ragStatus === 'AMBER' ? 'bg-yellow-100 text-yellow-800' :
ragStatus === 'RED' ? 'bg-red-100 text-red-800' : 'bg-gray-100
text-gray-600'; const icon = ragStatus === 'GREEN' ? '🟢' :
ragStatus === 'AMBER' ? '🟡' : ragStatus === 'RED' ? '🔴' : '⬜';
return ( <div className="relative" onMouseEnter={() =>
setShowDetail(true)} onMouseLeave={() => setShowDetail(false)} >
<div className={`px-2 py-1 rounded font-semibold text-right
${bgColor}`> {icon} {value.toFixed(2)}% </div> {/*
Tooltip */} {showTooltip && showDetail && ( <RAGTooltip
value={value} ragStatus={ragStatus} thresholds=
{thresholds} /> )} </div> ); }
// RAGBadge.tsx interface RAGBadgeProps { status: 'GREEN' | 'AMBER' |
'RED' | 'GRAY'; size?: 'sm' | 'md' | 'lg'; showLabel?: boolean; }
export function RAGBadge({ status, size = 'md', showLabel = true }:
RAGBadgeProps) { const sizeClasses = { sm: 'px-2 py-1 text-xs',
md: 'px-3 py-1.5 text-sm', lg: 'px-4 py-2 text-base' }; const
colorClasses = status === 'GREEN' ? 'bg-green-500 text-white'
: status === 'AMBER' ? 'bg-yellow-500 text-white' :
status === 'RED' ? 'bg-red-500 text-white' : 'bg-gray-
400 text-white'; const icon = status === 'GREEN' ? '✓' : status
=== 'AMBER' ? '⚡' : status === 'RED' ? '⚠' : '⦿'; const label =
status === 'GREEN' ? 'Excellent' : status === 'AMBER' ? 'Acceptable' :
status === 'RED' ? 'Poor' : 'No Data'; return ( <div className={`
inline-flex items-center justify-center gap-1 rounded font-bold
${sizeClasses[size]} ${colorClasses}`> <span>{icon}</span>
{showLabel && <span>{label}</span> </div> ); }
```

```

// RAGTooltip.tsx interface RAGTooltipProps {  value: number;  ragStatus:
string;  thresholds: {    green: number;    amber: number;    red:
number;  }; } export function RAGTooltip({ value, ragStatus, thresholds }:
RAGTooltipProps) {  return (    <div className="absolute z-50 top-full
left-0 mt-2 w-64 bg-white border-2 border-gray-300 rounded-lg shadow-xl p-
3">      {/* Header */}      <div className="font-bold text-sm mb-2 flex
items-center justify-between">        <span>GP ROI Performance</span>
        <RAGBadge status={ragStatus as any} size="sm" showLabel={false} />
      </div>      {/* Current Value */}      <div className="mb-3 p-2 bg-
gray-50 rounded">        <div className="text-xs text-gray-600">Current
ROI</div>        <div className="text-2xl font-bold">
{value.toFixed(2)}%        </div>        <div className="text-xs text-
gray-600">          Status: <span className="font-semibold">{ragStatus}<
span>        </div>        </div>        {/* Threshold Breakdown */}
<div className="space-y-2 mb-3">          <div className="text-xs font-
semibold text-gray-700">Thresholds:</div>          <div className=
{'flex items-center gap-2 ${value >= thresholds.green ? 'font-bold' :
''}}>            <div className="w-3 h-3 bg-green-500 rounded-full" />
            <span className="text-xs">≥ {thresholds.green}% = Green</span>
            {value >= thresholds.green && <span className="text-xs">← You are
here</span>}            </div>            <div className={'flex items-center gap-
2 ${value >= thresholds.amber && value < thresholds.green ? 'font-bold' :
''}}>              <div className="w-3 h-3 bg-yellow-500 rounded-full" />
              <span className="text-xs">{thresholds.amber}% - {thresholds.green}% =
Amber</span>              {value >= thresholds.amber && value <
thresholds.green && <span className="text-xs">← You are here</span>}
            </div>            <div className={'flex items-center gap-2 ${value <
thresholds.red ? 'font-bold' : ''}}>              <div className="w-3 h-3 bg-
red-500 rounded-full" />              <span className="text-xs">&lt;
{thresholds.red}% = Red</span>              {value < thresholds.red && <span
className="text-xs">← You are here</span>}            </div>            </div>
            {/* Visual Range */}            <div className="mb-3">              <div
className="text-xs font-semibold text-gray-700 mb-2">Visual Range:</div>
              <RAGRangeIndicator value={value} thresholds={thresholds} />
            </div>            {/* Guidance */}            <div className="text-xs text-gray-600
border-t pt-2">              {ragStatus === 'GREEN' && '✓ Excellent! This

```

```

'RED' && '⚠ Warning: Low ROI. Consider adjusting tactics or volume.')}
</div>    </div>    ); }
// RAGRangeIndicator.tsx interface RAGRangeIndicatorProps {  value:
number;  thresholds: {    green: number;    amber: number;    red:
number;  }; } export function RAGRangeIndicator({ value, thresholds }:
RAGRangeIndicatorProps) {  // Calculate percentages for visual display
// Assume range from 0% to green threshold + 10% for visual purposes
const maxRange = thresholds.green + 10;  const redWidth = (thresholds.red
/ maxRange) * 100;  const amberWidth = ((thresholds.green -
thresholds.red) / maxRange) * 100;  const greenWidth = 100 - redWidth -
amberWidth;  // Calculate position of indicator  const
indicatorPosition = Math.min((value / maxRange) * 100, 100);  return (
<div className="relative">    {/* Range Bar */}    <div
className="flex h-8 rounded overflow-hidden border border-gray-300">
      <div          className="bg-red-300 flex items-center justify-center
text-xs text-red-800"          style={{ width: `${redWidth}%` }}          >
          Red          </div>          <div          className="bg-yellow-
300 flex items-center justify-center text-xs text-yellow-800"
style={{ width: `${amberWidth}%` }}          >          Amber
        </div>          <div          className="bg-green-300 flex items-center
justify-center text-xs text-green-800"          style={{ width:
`${greenWidth}%` }}          >          Green          </div>          </div>
      {/* Current Value Indicator */}          <div          className="absolute
top-0 bottom-0 w-1 bg-black"          style={{ left: `${indicatorPosition}%`
}}          >          <div className="absolute -top-6 left-1/2 -translate-x-
1/2 text-xs font-bold whitespace-nowrap">          ▼ {value.toFixed(1)}%
        </div>          </div>          {/* Threshold Labels */}          <div
className="flex justify-between text-xs text-gray-600 mt-1">
<span>0%</span>          <span>{thresholds.red}%</span>          <span>
{thresholds.green}%</span>          <span>{maxRange}%</span>          </div>
      </div>    ); }
// FormattedValue.tsx (Helper component for displaying any KPI) interface
FormattedValueProps {  value: number | null;  format: 'number' |
'currency' | 'percentage' | 'text';  decimals?: number;  className?:
string; } export function FormattedValue({  value,  format,  decimals =
2,  className = '' }: FormattedValueProps) {  if (value === null || value

```

```

    case 'currency':      formatted = new Intl.NumberFormat('en-US', {
      style: 'currency',    currency: 'USD',
      minimumFractionDigits: decimals,    maximumFractionDigits: decimals
    }).format(value);      break;    case 'percentage':      formatted =
`${value.toFixed(decimals)}%`;      break;    case 'number':
formatted = new Intl.NumberFormat('en-US', {      minimumFractionDigits:
decimals,      maximumFractionDigits: decimals    }).format(value);
      break;    case 'text':      default:      formatted = String(value);
      break;    }    return <span className={className}>{formatted}</span>; }

```

PROVIDE:

1. All RAG component files

2. Interactive examples:

Click to show detailed modal with full explanation Hover to show tooltip Visual range indicator

3. Color system:

Green: #10B981 (bg), #065F46 (text) Amber: #F59E0B (bg), #92400E (text) Red: #EF4444 (bg), #991B1B (text) Gray: #9CA3AF (bg), #374151 (text)

4. Accessibility:

Use ARIA labels Color + icon (not just color) Keyboard navigation support

5. Example usage in grid:

```

// In SKU row <td className="sticky right-20"> <RAGCell      value=
{sku.calculated.GP_ROI_PCT}      ragStatus={sku.calculated.RAG_STATUS}
thresholds={{ green: 20, amber: 10, red: 10 }}    /> </td> <td
className="sticky right-0"> <RAGBadge      status=
{sku.calculated.RAG_STATUS}      size="sm"    /> </td>

```

6. Click to expand detail modal:

```

const [showModal, setShowModal] = useState(false); // On RAG cell click
<RAGDetailModal  isOpen={showModal}  onClose={() => setShowModal(false)}
  value={value}  ragStatus={ragStatus}  itemName={sku.sku_name}
recommendations={[      'Reduce CPP discount from 15% to 12%',      'Increase
planned volume to 5000 units',      'Remove visibility lumpsum spend'    ]}
/>

```

USE:

- React with TypeScript
- Tailwind CSS for styling
- Smooth transitions
- Mobile-friendly tooltips (tap instead of hover)

Create integration code and testing suite to ensure all components work together correctly.

CONTEXT: All individual pieces are built. Now need to:

1. Wire everything together
2. Test end-to-end workflows
3. Handle edge cases
4. Add error boundaries

REQUIREMENTS:

1. Main App Integration

Connect all hooks and components Initialize calculation engine on app start Handle authentication and permissions

2. Error Handling

Error boundaries for React components Graceful fallbacks for calculation errors Toast notifications for user feedback

3. Testing

Unit tests for calculation engine Integration tests for data flow E2E tests for user workflows

4. Performance Monitoring

Track calculation time Monitor re-renders Optimize bottlenecks

MAIN APP STRUCTURE:

```
// App.tsx import { useEffect } from 'react'; import { calculationEngine }
from './services/calculationEngine'; import { PlanningScreen } from
'./screens/PlanningScreen'; import { ErrorBoundary } from
'./components/ErrorBoundary'; import { Toaster } from
'./components/Toaster'; export function App() { // Initialize calculation
engine on app start  useEffect(() => {    const init = async () => {
  try {      console.log('🚀 Initializing calculation engine...');
  await calculationEngine.initialize();      console.log('✅ Calculation
engine ready');    } catch (error) {      console.error('❌ Failed to
initialize:', error);      // Show error toast    }    };
  init(); }, []);  return (    <ErrorBoundary>      <div
className="app">        <Toaster />        <PlanningScreen planId="plan-
123" />      </div>    </ErrorBoundary>  ); }

// screens/PlanningScreen.tsx interface PlanningScreenProps {  planId:
string; } export function PlanningScreen({ planId }: PlanningScreenProps) {
  const [planContext, setPlanContext] = useState({    channel: 'NKA',
category: 'Dairy'  });  return (    <div className="planning-screen">
```

```

        className="text-sm">                Channel: {planContext.channel} | Category:
{planContext.category}                    </div>                </header>                {/* Main Grid
*/}                <main className="p-4">                <PlanningGrid                planId=
{planId}                planContext={planContext}                />                </main>
</div>    ); }
// components/ErrorBoundary.tsx import React, { Component, ErrorInfo,
ReactDOM } from 'react'; interface Props {  children: ReactDOM; }
interface State {  hasError: boolean;  error: Error | null; } export
class ErrorBoundary extends Component<Props, State> {  constructor(props:
Props) {    super(props);    this.state = { hasError: false, error: null
};  }  static getDerivedStateFromError(error: Error): State {    return
{ hasError: true, error };  }  componentDidCatch(error: Error, errorInfo:
ErrorInfo) {    console.error('Error caught by boundary:', error,
errorInfo);    // Log to error tracking service (Sentry, etc.)  }
render() {    if (this.state.hasError) {      return (        <div
className="error-screen p-8 text-center">          <h1 className="text-2xl
font-bold text-red-600 mb-4">            Something went wrong
          </h1>          <p className="text-gray-600 mb-4">
{this.state.error?.message}          </p>          <button
onClick={() => window.location.reload()}          className="px-4 py-2
bg-blue-500 text-white rounded hover:bg-blue-600"          >
Reload Page          </button>          </div>        );    }    return
this.props.children;  } }

```

UNIT TESTS:

```

// __tests__/formulaParser.test.ts import { formulaParser } from
'../services/formulaParser'; describe('FormulaParser', () => {
describe('extractDependencies', () => {  it('should extract variable
names from formula', () => {    const formula = "PLANNED_VOL -
BASE_VOL";    const deps = formulaParser.extractDependencies(formula);
    expect(deps).toEqual(['PLANNED_VOL', 'BASE_VOL']);  });
it('should handle complex formulas', () => {    const formula = "
(INCR_GP / INCR_SPEND) * 100";    const deps =
formulaParser.extractDependencies(formula);
    expect(deps).toEqual(['INCR_GP', 'INCR_SPEND']);  });  it('should
exclude function names', () => {    const formula = "IF(GP_ROI >= 20,
'GREEN', 'RED')";    const deps =

```

```

describe('parseExpression', () => {
  it('should calculate simple subtraction', () => {
    const parsed = formulaParser.parseFormula("PLANNED_VOL - BASE_VOL", "expression");
    const result = parsed.execute({ PLANNED_VOL: 4200, BASE_VOL: 3200 });
    expect(result).toBe(1000);
  });
  it('should calculate percentage', () => {
    const parsed = formulaParser.parseFormula("(INCR_VOL / BASE_VOL) * 100", "expression");
    const result = parsed.execute({ INCR_VOL: 1000, BASE_VOL: 3200 });
    expect(result).toBeCloseTo(31.25, 2);
  });
  it('should handle division by zero', () => {
    const parsed = formulaParser.parseFormula("100 / ZERO_VALUE", "expression");
    const result = parsed.execute({ ZERO_VALUE: 0 });
    expect(result).toBeNull();
  });
  it('should handle missing dependencies', () => {
    const parsed = formulaParser.parseFormula("PLANNED_VOL - BASE_VOL", "expression");
    const result = parsed.execute({ PLANNED_VOL: 4200 }); // BASE_VOL missing
    expect(result).toBeNull();
  });
});
describe('parseConditional', () => {
  it('should evaluate simple IF', () => {
    const parsed = formulaParser.parseFormula("IF(GP_ROI >= 20, 'GREEN', 'RED')", "conditional");
    expect(parsed.execute({ GP_ROI: 25 })).toBe('GREEN');
    expect(parsed.execute({ GP_ROI: 15 })).toBe('RED');
  });
  it('should evaluate nested IF', () => {
    const parsed = formulaParser.parseFormula("IF(GP_ROI >= 20, 'GREEN', IF(GP_ROI >= 10, 'AMBER', 'RED'))", "conditional");
    expect(parsed.execute({ GP_ROI: 25 })).toBe('GREEN');
    expect(parsed.execute({ GP_ROI: 15 })).toBe('AMBER');
    expect(parsed.execute({ GP_ROI: 5 })).toBe('RED');
  });
});
describe('validateFormula', () => {
  it('should validate correct formula', () => {
    const result = formulaParser.validateFormula("PLANNED_VOL - BASE_VOL", "expression");
    expect(result.isValid).toBe(true);
    expect(result.errors).toHaveLength(0);
  });
  it('should detect invalid syntax', () => {
    const result = formulaParser.validateFormula("PLANNED_VOL - -", "expression");
    expect(result.isValid).toBe(false);
    expect(result.errors.length).toBeGreaterThan(0);
  });
});
// __tests__/calculationEngine.test.ts import { calculationEngine } from
'../services/calculationEngine'; describe('CalculationEngine', () => {

```

```

describe('calculateSKU', () => {
  it('should calculate all KPIs for a SKU', () => {
    const context = {
      planned_volume: 4200,
      base_volume: 3200,
      list_price: 4.00,
      cogs_per_piece: 1.80,
      tactic_values: {
        CPP_ON_PCT: 10
      }
    };
    const results = calculationEngine.calculateSKU(context);
    expect(results.INCR_VOL.value).toBe(1000);
    expect(results.VOL_UPLIFT_PCT.value).toBeCloseTo(31.25, 2);
    expect(results.BASE_GSV.value).toBe(12800);
    expect(results.PLANNED_GSV.value).toBe(16800);
  });
  it('should handle zero base volume', () => {
    const context = {
      planned_volume: 5000,
      base_volume: 0, // New product
      list_price: 4.00,
      cogs_per_piece: 1.80
    };
    const results = calculationEngine.calculateSKU(context);
    expect(results.INCR_VOL.value).toBe(5000);
    expect(results.VOL_UPLIFT_PCT.value).toBeNull(); // Can't calculate uplift
  });
});
describe('recalculate', () => {
  it('should only recalculate affected KPIs', () => {
    const initialContext = {
      PLANNED_VOL: 4200,
      BASE_VOL: 3200,
      BPTT: 4.00,
      INCR_VOL: 1000,
      BASE_GSV: 12800,
      PLANNED_GSV: 16800
    };
    const updates = calculationEngine.recalculate(
      'PLANNED_VOL',
      5000,
      initialContext
    ); // Should recalculate INCR_VOL and dependent KPIs
    expect(updates).toHaveProperty('INCR_VOL');
    expect(updates).toHaveProperty('PLANNED_GSV');
    expect(updates).toHaveProperty('VOL_UPLIFT_PCT'); // Should NOT recalculate independent KPIs
    expect(updates).not.toHaveProperty('BASE_VOL');
    expect(updates).not.toHaveProperty('BASE_GSV');
  });
});
describe('aggregateFU', () => {
  it('should sum SKU values to FU level', () => {
    const skuResults = [
      { INCR_VOL: { value: 1000 }, PLANNED_GP: { value: 2500 } },
      { INCR_VOL: { value: 750 }, PLANNED_GP: { value: 1800 } },
      { INCR_VOL: { value: 500 }, PLANNED_GP: { value: 1200 } }
    ];
    const fuResults = calculationEngine.aggregateFU(skuResults);
    expect(fuResults.INCR_VOL).toBe(2250); // 1000 + 750 + 500
    expect(fuResults.PLANNED_GP).toBe(5500); // 2500 + 1800 + 1200
  });
});

```

```

value: 25.5 } }, { INCR_VOL: { value: 750 }, GP_MARGIN_PCT: { value: 30.2 } }, { INCR_VOL:
{ value: 500 }, GP_MARGIN_PCT: { value: 22.8 } } ];
const fuResults = calculationEngine.aggregateFU(skuResults);
expect(fuResults.INCR_VOL).toBe(2250); // sum
expect(fuResults.GP_MARGIN_PCT).toBeCloseTo(26.17, 2); // avg });
});
describe('formatValue', () => { it('should format currency', () => { const kpi = {
display_format: 'currency', decimal_places: 2 }; const formatted =
calculationEngine['formatValue'](12500.50, kpi); expect(formatted).toBe('$12,500.50'); });

it('should format percentage', () => {
const kpi = { display_format: 'percentage', decimal_places: 2 };
const formatted = calculationEngine['formatValue'](25.456, kpi);
expect(formatted).toBe('25.46%');
});

it('should format number with thousand separator', () => {
const kpi = { display_format: 'number', decimal_places: 0 };
const formatted = calculationEngine['formatValue'](123456, kpi);
expect(formatted).toBe('123,456');
});
}); });

```

```

```typescript

```

```

// __tests__/calculationGraph.test.ts

```

```

import { calculationGraph } from '../services/calculationGraph';

```

```

describe('CalculationGraph', () => {
const sampleKPIs = [
{ kpi_code: 'PLANNED_VOL', depends_on_kpis: [] },
{ kpi_code: 'BASE_VOL', depends_on_kpis: [] },
{ kpi_code: 'BPTT', depends_on_kpis: [] },
{ kpi_code: 'INCR_VOL', depends_on_kpis: ['PLANNED_VOL', 'BASE_VOL'] },
{ kpi_code: 'BASE_GSV', depends_on_kpis: ['BASE_VOL', 'BPTT'] },
{ kpi_code: 'PLANNED_GSV', depends_on_kpis: ['PLANNED_VOL', 'BPTT'] },
{ kpi_code: 'INCR_GSV', depends_on_kpis: ['PLANNED_GSV', 'BASE_GSV'] },
{ kpi_code: 'VOL_UPLIFT_PCT', depends_on_kpis: ['INCR_VOL', 'BASE_VOL']
}

```

```
];
```

```
beforeEach(async () => {
 await calculationGraph.buildGraph(sampleKPIs);
});
```

```
describe('buildGraph', () => {
 it('should build graph with correct nodes', () => {
 const order = calculationGraph.getCalculationOrder();
 expect(order).toContain('PLANNED_VOL');
 expect(order).toContain('INCR_VOL');
 expect(order).toContain('INCR_GSV');
 });
```

```
 it('should calculate correct levels', () => {
 const plannedVolNode = calculationGraph.getNode('PLANNED_VOL');
 const incrVolNode = calculationGraph.getNode('INCR_VOL');
 const incrGsvNode = calculationGraph.getNode('INCR_GSV');

 expect(plannedVolNode?.level).toBe(0);
 expect(incrVolNode?.level).toBe(1);
 expect(incrGsvNode?.level).toBe(3); // Depends on PLANNED_GSV and
 BASE_GSV (level 2)
 });
});
```

```
describe('getCalculationOrder', () => {
 it('should return topologically sorted order', () => {
 const order = calculationGraph.getCalculationOrder();
 // PLANNED_VOL and BASE_VOL should come before INCR_VOL
 const plannedIdx = order.indexOf('PLANNED_VOL');
 const baseIdx = order.indexOf('BASE_VOL');
 const incrIdx = order.indexOf('INCR_VOL');

 expect(plannedIdx).toBeLessThan(incrIdx);
 expect(baseIdx).toBeLessThan(incrIdx);
 });
```

```
 it('should calculate dependencies before dependents', () => {
 const order = calculationGraph.getCalculationOrder();
```

```

const baseGsvIdx = order.indexOf('BASE_GSV');
const plannedGsvIdx = order.indexOf('PLANNED_GSV');
const incrGsvIdx = order.indexOf('INCR_GSV');

expect(baseGsvIdx).toBeLessThan(incrGsvIdx);
expect(plannedGsvIdx).toBeLessThan(incrGsvIdx);
});
});

describe('getAffectedKPIs', () => {
 it('should return all dependent KPIs when input changes', () => {
 const affected = calculationGraph.getAffectedKPIs('PLANNED_VOL');
 expect(affected).toContain('INCR_VOL');
 expect(affected).toContain('PLANNED_GSV');
 expect(affected).toContain('VOL_UPLIFT_PCT');
 expect(affected).toContain('INCR_GSV');
 });

 it('should not include the changed KPI itself', () => {
 const affected = calculationGraph.getAffectedKPIs('PLANNED_VOL');
 expect(affected).not.toContain('PLANNED_VOL');
 });

 it('should not include independent KPIs', () => {
 const affected = calculationGraph.getAffectedKPIs('PLANNED_VOL');
 expect(affected).not.toContain('BASE_VOL');
 expect(affected).not.toContain('BASE_GSV');
 });

 it('should return empty array for leaf nodes', () => {
 const affected = calculationGraph.getAffectedKPIs('INCR_GSV');
 expect(affected).toHaveLength(0);
 });
});

describe('circular dependency detection', () => {
 it('should throw error for circular dependencies', async () => {
 const circularKPIs = [
 { kpi_code: 'A', depends_on_kpis: ['B'] },
 { kpi_code: 'B', depends_on_kpis: ['C'] },

```

```
];
```

```
await
expect(calculationGraph.buildGraph(circularKPIs)).rejects.toThrow('Circular dependency');
});
});
});
````
```

INTEGRATION TEST:

```
````typescript  
// __tests__/integration/planCalculations.test.tsx
import { renderHook, act, waitFor } from '@testing-library/react';
import { usePlanCalculations } from '../../hooks/usePlanCalculations';

describe('Plan Calculations Integration', () => {
 it('should calculate KPIs when SKU volume changes', async () => {
 const { result } = renderHook(() => usePlanCalculations('test-plan-1'));

 // Wait for initial load
 await waitFor(() => {
 expect(result.current.fus.length).toBeGreaterThan(0);
 });

 const initialGrandTotal = result.current.grandTotals.GP_ROI_PCT;

 // Update SKU volume
 act(() => {
 result.current.updateSKUVolume('fu-1', 'sku-1', 5000);
 });

 // Should recalculate
 await waitFor(() => {
 expect(result.current.grandTotals.GP_ROI_PCT).not.toBe(initialGrandTotal);
 });

 // Check that calculations cascaded
 const fu = result.current.fus.find(f => f.fu_id === 'fu-1');
```

```

expect(sku?.planned_volume).toBe(5000);
expect(sku?.calculated.INCR_VOL).toBeDefined();
expect(sku?.calculated.GP_ROI_PCT).toBeDefined();
});

it('should aggregate SKU values to FU level', async () => {
 const { result } = renderHook(() => usePlanCalculations('test-plan-1'));

 await waitFor(() => {
 expect(result.current.fus.length).toBeGreaterThan(0);
 });

 const fu = result.current.fus[0];
 // FU planned volume should be sum of SKU volumes
 const expectedTotal = fu.skus.reduce((sum, sku) => sum +
 sku.planned_volume, 0);
 expect(fu.calculated.PLANNED_VOL).toBe(expectedTotal);
});

it('should recalculate all SKUs when FU tactic changes', async () => {
 const { result } = renderHook(() => usePlanCalculations('test-plan-1'));

 await waitFor(() => {
 expect(result.current.fus.length).toBeGreaterThan(0);
 });

 const fu = result.current.fus[0];
 const initialSKU GP = fu.skus[0].calculated.PLANNED_GP;

 // Update FU tactic
 act(() => {
 result.current.updateFUTactic('fu-1', 'CPP_ON_PCT', 15); // Increase
 from 10% to 15%
 });

 // All SKUs should recalculate
 await waitFor(() => {
 const updatedFU = result.current.fus.find(f => f.fu_id === 'fu-1');
 const updatedSKU = updatedFU?.skus[0];
 });

```

```

});
});

it('should auto-save after debounce period', async () => {
 jest.useFakeTimers();
 const { result } = renderHook(() => usePlanCalculations('test-plan-1'));

 await waitFor(() => {
 expect(result.current.fus.length).toBeGreaterThan(0);
 });

 // Make change
 act(() => {
 result.current.updateSKUVolume('fu-1', 'sku-1', 5000);
 });

 // Fast-forward past debounce (2 seconds)
 act(() => {
 jest.advanceTimersByTime(2500);
 });

 // Should have triggered save (check console or mock Supabase)
 // This would require mocking Supabase client

 jest.useRealTimers();
});
});
```

```

E2E TEST (using Playwright or Cypress):

```

```typescript
// e2e/planning-grid.spec.ts
import { test, expect } from '@playwright/test';

test.describe('Planning Grid', () => {
 test.beforeEach(async ({ page }) => {
 await page.goto('/plan/test-plan-1');
 await page.waitForSelector('.planning-grid-table');
 });
});

```

```

const grandTotals = page.locator('.grand-totals-panel');
await expect(grandTotals).toBeVisible();
// Check for key metrics
await expect(grandTotals.locator('text=VOLUME')).toBeVisible();
await expect(grandTotals.locator('text=GP ROI')).toBeVisible();
});

test('should expand/collapse FU rows', async ({ page }) => {
// Initially collapsed
const skuRow = page.locator('tr.sku-row').first();
await expect(skuRow).not.toBeVisible();

// Click expand button
await page.locator('button:has-text("+ ")').first().click();

// SKU rows should now be visible
await expect(skuRow).toBeVisible();
});

test('should update calculations when volume changes', async ({ page })
=> {
// Expand first FU
await page.locator('button:has-text("+ ")').first().click();

// Get initial ROI value
const roiCell = page.locator('tr.sku-row').first().locator('td.rag-
cell');
const initialRoi = await roiCell.textContent();

// Change planned volume
const volumeInput = page.locator('tr.sku-
row').first().locator('input[type="text"]').first();
await volumeInput.click();
await volumeInput.fill('5000');
await volumeInput.blur();

// Wait for calculation
await page.waitForTimeout(500);

// ROI should have changed

```

```

expect(updatedRoi).not.toBe(initialRoi);

// Grand totals should update
const grandTotalRoi = page.locator('.grand-totals-panel').locator('text=/GP ROI.%%/');
await expect(grandTotalRoi).toBeVisible();
});

test('should show RAG tooltip on hover', async ({ page }) => {
 await page.locator('button:has-text("⛶")').first().click();
 const ragCell = page.locator('tr.sku-row').first().locator('.rag-cell');
 await ragCell.hover();

 // Tooltip should appear
 const tooltip = page.locator('.rag-tooltip');
 await expect(tooltip).toBeVisible();
 await expect(tooltip.locator('text=Thresholds')).toBeVisible();
});

test('should highlight changes with animation', async ({ page }) => {
 await page.locator('button:has-text("⛶")').first().click();

 const volumeInput = page.locator('tr.sku-row').first().locator('input[type="text"]').first();
 await volumeInput.click();
 await volumeInput.fill('6000');
 await volumeInput.blur();

 // Should have highlight class during animation
 const animatedValue = page.locator('tr.sku-row').first().locator('.bg-green-200');
 await expect(animatedValue).toBeVisible();

 // Highlight should fade after animation
 await page.waitForTimeout(1000);
 await expect(animatedValue).not.toBeVisible();
});

test('should validate input ranges', async ({ page }) => {
 await page.locator('button:has-text("⛶")').first().click();

```

```

// Try to enter negative volume
const volumeInput = page.locator('tr.sku-
row').first().locator('input[type="text"]').first();
await volumeInput.click();
await volumeInput.fill('-100');
await volumeInput.blur();

// Should show error
const errorMessage = page.locator('text=/Minimum/');
await expect(errorMessage).toBeVisible();
});

test('should show only applicable tactics for plan context', async ({
page }) => {
// Check that only NKA-applicable tactics are shown
const header = page.locator('thead');
// Should show CPP On-Invoice (applies to NKA)
await expect(header.locator('text=CPP On-Invoice')).toBeVisible();
// Should NOT show WS TPR (only for Wholesale)
await expect(header.locator('text=WS TPR')).not.toBeVisible();
});
});
```

```

PERFORMANCE MONITORING:

```

```typescript
// utils/performanceMonitor.ts
export class PerformanceMonitor {
private metrics: Map<string, number[]> = new Map();

startMeasure(label: string): () => void {
const startTime = performance.now();
return () => {
const endTime = performance.now();
const duration = endTime - startTime;
if (!this.metrics.has(label)) {
this.metrics.set(label, []);
}
this.metrics.get(label)!.push(duration);
if (duration > 500) {

```

```

console.warn(`⚠ Slow operation: ${label} took
${duration.toFixed(2)}ms`);
} else {
console.log(`🕒 ${label}: ${duration.toFixed(2)}ms`);
}
};
}

```

```

getStats(label: string) {
const values = this.metrics.get(label) || [];
if (values.length === 0) return null;

```

```

const sum = values.reduce((a, b) => a + b, 0);
const avg = sum / values.length;
const min = Math.min(...values);
const max = Math.max(...values);

```

```

return { avg, min, max, count: values.length };
}

```

```

logAllStats() {
console.table(
Array.from(this.metrics.keys()).map(label => ({
operation: label,
...this.getStats(label)
})))
);
}
}

```

```

export const perfMonitor = new PerformanceMonitor();

```

```

// Usage in calculation engine
async calculateSKU(context: CalculationContext) {
const endMeasure = perfMonitor.startMeasure('calculateSKU');
try {
// ... calculation logic
return results;
} finally {
endMeasure();
}
}

```

```
}
```
```

TOAST NOTIFICATION SYSTEM:

```
```typescript  
// components/Toaster.tsx
import { useState, useEffect } from 'react';
import { toast } from 'react-hot-toast';

export function Toaster() {
 return (
 <div className="toaster-container">
 /* Use react-hot-toast or custom implementation */
 </div>
);
}

// Usage in components
import { toast } from 'react-hot-toast';

// Success
toast.success('Plan saved successfully');

// Error
toast.error('Failed to calculate KPIs');

// Warning
toast.warning('Some items have negative ROI');

// Info
toast('Calculation completed', { icon: '🔍' });
```
```

PROVIDE:

1. ****Complete integration files****
2. ****All test files**** (unit, integration, e2e)
3. ****Test coverage report**** showing >80% coverage
4. ****Performance benchmarks:****
 - Initial load: < 2s

- FU tactic update: < 300ms
- Save to database: < 1s

5. **Error handling examples:**

- Network errors (retry logic)
- Calculation errors (fallback to null)
- Permission errors (redirect to access denied)

6. **CI/CD configuration** (GitHub Actions):

```
```yaml
name: Test
on: [push, pull_request]
jobs:
 test:
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v2
 - uses: actions/setup-node@v2
 - run: npm ci
 - run: npm run test
 - run: npm run test:e2e
 - run: npm run build
```
```

USE:

- Jest for unit tests
- React Testing Library for integration tests
- Playwright or Cypress for E2E tests
- TypeScript strict mode
- 80%+ code coverage requirement

SUMMARY: Complete Prompt Sequence

You now have **10 sequential prompts** to build the dynamic TPM calculation system:

1.  **Database Schema** - Foundation for dynamic KPIs
2.  **Formula Parser** - Parse and execute formula strings
3.  **Calculation Graph** - Dependency management
4.  **Calculation Engine** - Core calculation logic
5.  **React Hook** - State management and real-time updates

6.  **Dynamic Columns** - Context-aware column configuration
7.  **Grid Components** - Planning grid UI with FU/SKU hierarchy
8.  **Grand Totals & Animation** - Visual feedback components
9.  **RAG Status Components** - Performance visualization
10.  **Integration & Testing** - Tie everything together

Each prompt:

- Is focused and buildable independently
- Has clear dependencies on previous prompts
- Includes examples and test cases
- Can be used with AI code generators (Claude, ChatGPT, Gemini)

Recommended approach:

1. Build prompts 1-5 first (backend/logic)
2. Test thoroughly before moving to UI
3. Build prompts 6-9 (frontend)
4. Complete with prompt 10 (integration)

Would you like me to create additional prompts for:

- Admin screens for KPI configuration UI?
- Budget tracking and alerting?
- Export functionality?
- Mobile responsive design?